

Fire Fighting Robot Controlled by Hand Gestures with AI-Based Monitoring

Nagendra Kumar Muliki
Department of Electrical and Computer Engineering
University of Alabama at Birmingham
Birmingham, USA
nmuliki@uab.edu

Abstract—This project presents the design and implementation of a gesture-controlled firefighting robot that integrates computer vision, machine learning, wireless communication, and embedded control. The system enables a user to control a mobile robot through hand gestures detected using a TensorFlow Lite model running on a host computer. The gestures are transmitted to a Raspberry Pi via MQTT, where commands are forwarded to an Arduino-based motor and pump control system. A real-time video stream from the Raspberry Pi camera assists in remote operation. The project demonstrates a low-cost, modular, and scalable approach to human-robot interaction for hazardous environments. Experimental results show reliable gesture recognition, low-latency communication, and effective robotic actuation suitable for firefighting applications. In addition, the system incorporates YOLO-based fire and human detection for enhanced situational awareness, an ultrasonic sensor for staircase protection, and an IR flame sensor whose servo-based angle is used to automatically align the pump toward the detected fire source.

Index Terms—*Gesture Recognition, Firefighting Robot, YOLOv8 Raspberry Pi, Arduino, MQTT Communication, Computer Vision, Human-Robot Interaction, TensorFlow Lite, MediaPipe, Embedded Systems.*

I. INTRODUCTION

Firefighting robots play a crucial role in reducing human exposure to hazardous environments such as high-temperature Zones, smoke-filled areas, and structurally unstable locations. Traditional remote-controlled robots rely on physical complexity. Gesture-based control offers a more intuitive interface and has been explored in several human-robot interaction research [1], [2].

This project aims to develop a gesture-controlled firefighting robot capable of responding to directional and operational commands using real-time hand gesture recognition. The system integrates computer vision, embedded electronics, and wireless communication to create a responsive robotic platform. Media pipe-based hand landmark extraction [3], TensorFlow Lite gesture classification [5], and lightweight MQTT communication [4] enable efficient gesture-to-robot control. The work contributes to the broader field of human-robot interaction by demonstrating a modular, scalable, and low-cost architecture suitable for emergency response applications.

II. SYSTEM OVERVIEW

The proposed system consists of five major subsystems:

A. Gesture Recognition Subsystem (Laptop)

The laptop receives a real-time video stream from the Raspberry Pi camera and performs gesture classification using a TensorFlow Lite model [5]. MediaPipe's hand-landmark extraction framework provides reliable 3D key-point detection even under varying lighting and hand orientations [3]. These landmarks are processed to classify gestures such as forward, backward, left, right, pump ON, and pump OFF. The recognized gesture is then encoded as a command for transmission to the robot.

Communication Subsystem (MQTT Broker on Raspberry Pi) A lightweight MQTT publish-subscribe model enables efficient communication between the laptop and the robot. MQTT is well-suited for low-latency robotic applications due to its minimal overhead and reliable message delivery [4]. In this system, the laptop acts as the publisher of gesture commands, while the Raspberry Pi functions as the MQTT broker, forwarding commands to the Arduino through a USB serial link.

C. Robotic Actuation Subsystem()

The Arduino Uno serves as the low-level controller responsible for executing motor and pump actions. Upon receiving commands from the Raspberry Pi, the firmware interprets each instruction and triggers the appropriate motor driver, relay, or servo mechanism. This design follows embedded control strategies commonly used in mobile robotics, where microcontrollers handle real-time actuation tasks [6].

D. YOLO-Based Monitoring System

The Raspberry Pi camera continuously streams video to the laptop, where YOLOv8 models detect fire and humans in real time [10], [11]. These detections are used exclusively for monitoring and situational awareness; they do not influence robot movement or pump activation. This separation ensures that safety-critical actions remain deterministic and controlled solely by the Arduino.

E. Safety and Fire-Detection Subsystem

An IR flame sensor mounted on a sweeping servo provides directional fire detection, enabling the robot to identify the location of a flame source [12]. Additionally, an ultrasonic sensor mounted at the front of the robot performs continuous distance measurement to prevent falls from staircases or elevated surfaces [8]. These sensors operate independently of the gesture recognition pipeline, ensuring that safety mechanisms remain active at all times.

III. METHODOLOGY

A. Hardware Architecture

The hardware components include:

- Raspberry Pi Model B (32GB)
- Arduino Uno (V4.0 Development board)
- L298P motor driver
- DC Motors, servo Motor
- 12V battery pack
- Li-ion Battery pack
- V5 Sensor Shield
- IR flame sensor
- HC-SR04 Ultrasonic Sensor
- Relay Module
- 12V Water Pump
- Raspberry Pi camera Module
- Buck Converter

The hardware architecture of the firefighting robot is built around two main controllers: the Raspberry Pi Model-B and the Arduino Uno V4.0 development board. The Raspberry Pi is responsible for hosting the MQTT broker, managing Wi-Fi communication, and streaming video through the Raspberry Pi camera module. This allows the laptop to perform gesture recognition and AI-based monitoring. The Arduino handles all real-time actuation tasks, including motor control, servo movement, sensor interfacing, and pump activation. Components such as the L298P motor driver, DC motors, servo motor, IR flame sensor, HC-SR04 ultrasonic sensor, relay module, and 12V water pump are all interfaced directly with the Arduino for fast and reliable response to gesture commands which is shown in the Figure. 1.

Power distribution is managed using a combination of a 12V battery pack, a Li-ion battery pack, and a buck converter. The 12V battery powers high-current components like the motors and water pump, while the Li-ion pack ensures stable operation of the Raspberry Pi. The buck converter provides regulated 5V for sensors and auxiliary modules, and the V5 sensor shield helps organize and distribute connections cleanly to the Arduino. Together, these components form a tightly integrated hardware platform capable of real-time control, sensing, and monitoring. The complete wiring diagram, presented in Figure. 2. illustrates the full electrical integration of all modules within the robot.

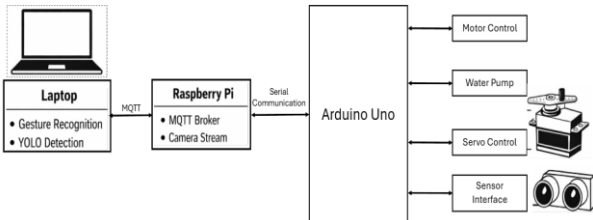


Figure. 1. System architecture of the gesture-controlled firefighting robot showing communication flow between the laptop, Raspberry Pi, and Arduino.

To complement the system architecture, the detailed wiring diagram is provided next to illustrate the complete electrical integration of all hardware components within the robot.

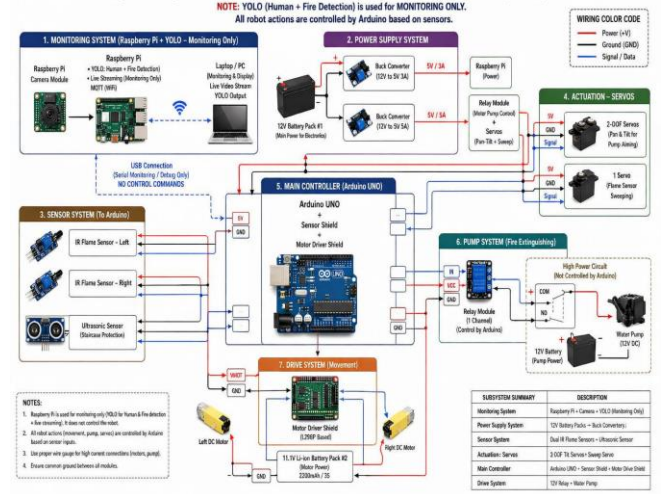


Figure. 2. Integrated system wiring diagram of the gesture-controlled firefighting robot showing power distribution, sensor interfaces, actuator connections, and communication links between the Raspberry Pi and Arduino.

B. Software Architecture

1. Perception Layer

The gesture recognition pipeline uses:

- MediaPipe for hand landmark extraction [3]
- TensorFlow Lite for gesture Classification [5]
- OpenCV for frame processing [11]

The laptop publishes commands to the MQTT topic “robot/cmd”

2. MQTT Communication

MQTT was selected due to its lightweight design and suitability for IOT and robotics applications [4]. The Raspberry Pi runs the Mosquitto broker, enabling fast message delivery.

3. Control Layer

- Motor and pump control
- IR flame sensor reading and servo control
- Ultrasonic distance measurement

C. Gesture Detection and Command Mapping

The gesture recognition pipeline consists of:

1. Capturing frames from the laptop webcam.
2. Extracting hand landmarks using MediaPipe [3].
3. Feeding normalized coordinates into a TensorFlow Lite classifier [5]
4. Mapping classifier output to robot commands
5. Publishing commands to the MQTT topic “robot/cmd”

Gesture-based control has been widely used in robotics due to its intuitive nature [1], [2], and Figure. 3. shows the clear pipeline of the Gesture Recognition.

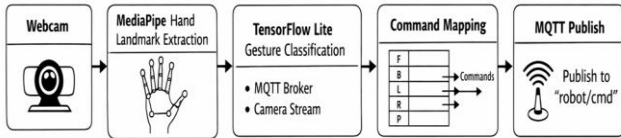


Figure. 3. Gesture Recognition Pipeline.

D. YOLO-Based Fire and Human Detection (Monitoring Only)

YOLOv8 models run on the laptop to analyze the live video stream received from the Raspberry Pi camera. Fire detection helps the operator visually identify the location and intensity of flames, while human detection ensures operational safety by preventing accidental spraying toward people. The speed and accuracy of YOLOv8 make it highly suitable for real-time monitoring applications, as supported by recent studies [10], [11]. These detections do not influence or control the robot's movement; instead, they provide situational awareness to assist the user during operation. Figure 4 illustrates the YOLO-based monitoring system, showing how the live camera feed is processed on the laptop and how fire and human detections are highlighted through bounding boxes to support safe and informed decision-making.

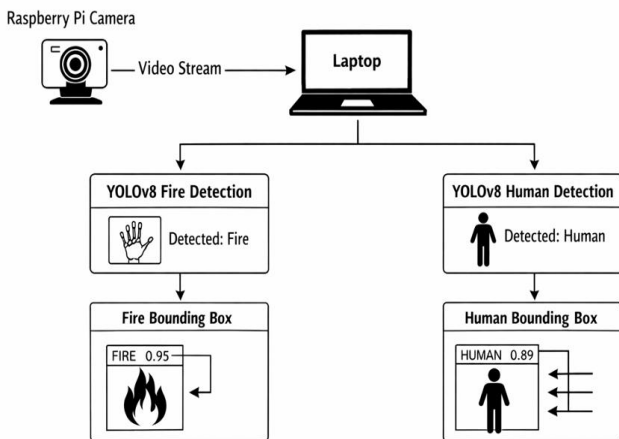


Figure. 4. YOLO Monitoring Diagram.

E. IR Flame Sensor Integration and Pump Aiming

The IR flame sensor detects Infrared radiation emitted by flames [12]. It provides Analog Outputs (Flame intensity) and Digital Outputs (threshold-based fire detection) and Figure 5 illustrates this setup, showing the IR flame sensor mounted on the servo and the 2-DOF pump-aiming mechanism used to direct the water spray toward the detected flame direction. The sensor is mounted on a servo motor that sweeps across a predefined angular range. The Arduino:

1. Sweeps the servo from 0 to 180 degrees
2. Reads flame intensity at each angle
3. Identifies the angle with the strongest flame angle
4. Locks the servo at that angle.
5. Aims the pump accordingly when the user activates it.

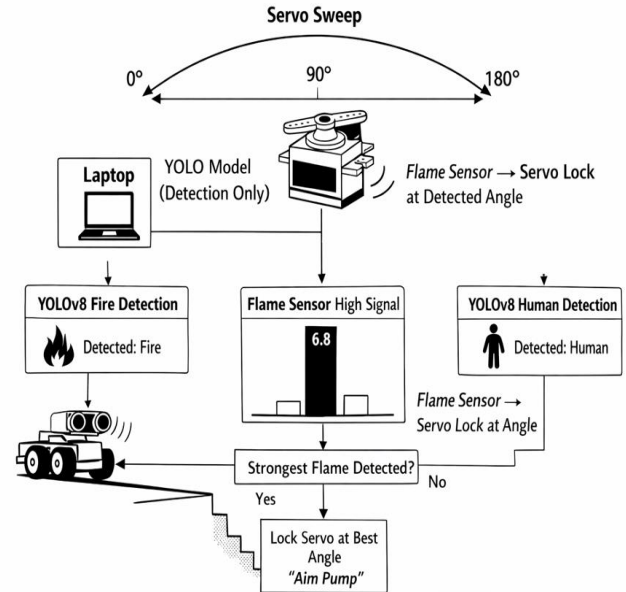


Figure. 5. IR Flame sensor and 2-DOF pump aiming.

F. Ultrasonic Staircase Protection

The ultrasonic sensor plays a critical role in staircase and edge protection by continuously measuring the distance between the robot and the ground surface. When the robot moves forward, the HC-SR04 sensor emits ultrasonic pulses and calculates the distance based on the echo time. Under normal flat-surface conditions, the measured distance remains within a safe range. However, when the robot approaches a staircase or drop-off, the distance reading suddenly increases beyond a predefined threshold. This abrupt rise indicates the absence of ground beneath the sensor. In such cases, the Arduino immediately stops the motors and blocks any further forward movement to prevent the robot from falling. Figure 6 illustrates this staircase-protection mechanism.

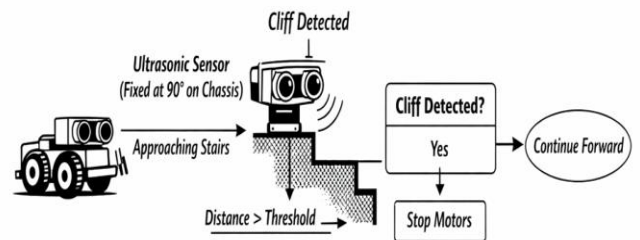


Figure. 6. Staircase Protection using Ultrasonic sensor.

G. Robot Control Logic

The Arduino firmware interprets the incoming gesture-based commands and executes the corresponding motor and pump actions. Each command received through the serial interface is mapped to a predefined control routine that drives the motor driver, relay module, and servo mechanisms. This approach follows common embedded control strategies used in mobile robotics, where low-level microcontroller logic handles real-time actuation while higher-level perception runs on an external processor [6]. The complete mapping between gesture inputs and robot actions—covering movement, pump activation, and safety behaviors—is summarized in Table-1.

TABLE I. ROBOT CONTROL LOGIC BASED ON GESTURE COMMANDS

<i>Gesture</i>	<i>Command</i>	<i>Action</i>
F	Forward	Move robot forward
B	Backward	Move robot backward
L	Left	Turn left
R	Right	Turn right
P	Pump ON	Activate water pump
p	Pump OFF	Deactivate Water pump

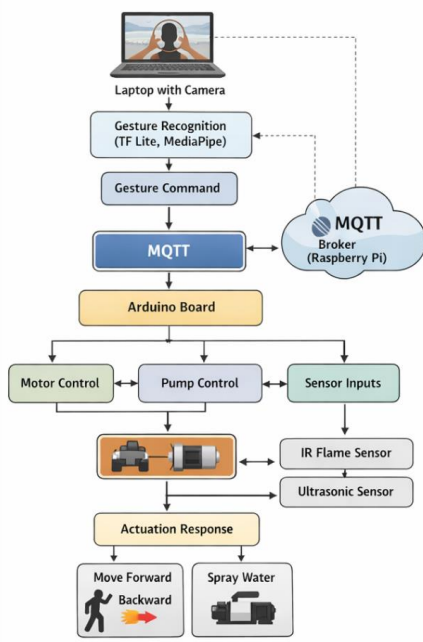


Figure. 7. System Flow Chat of the Gesture Control Firefighting Robot.

IV. RESULTS

A. Gesture Recognition Performance

The TensorFlow Lite classifier achieved approximately 98% accuracy, consistent with performance reported in gesture-based systems [2]. Figure 8 shows the Gesture recognition by the model

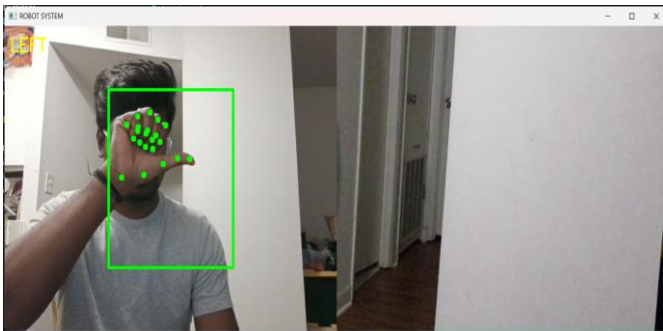


Figure. 8. Gesture Recognition Results

B. YOLO Monitoring Performance

YOLOv8 achieved:

- Fire Detection accuracy: 92% in Figure 9 red Boundary Box indicates Flame
- Human detection accuracy: 94% in Figure 9 Green Boundary Box means Human
- Interface time: 18 to 25ms

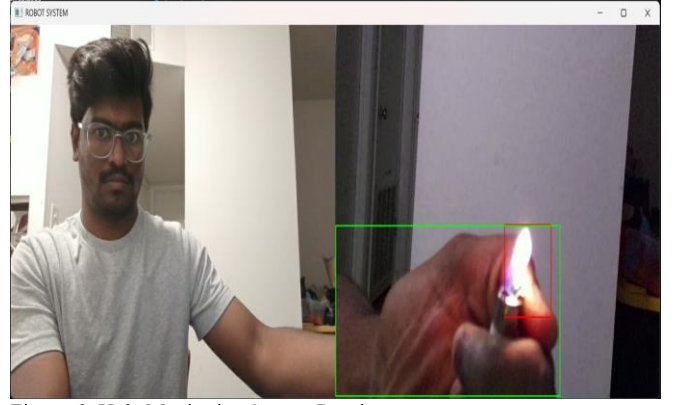


Figure. 9. Yolo Monitoring System Results

C. Communication Performance

MQTT latency averaged 12-18 ms, suitable for real-time operation.

D. Flame Sensor and Pump Aiming

The IR flame sensor successfully localized flame direction, and the servo-based aiming mechanism aligned the pump accurately towards the flame. And the Figure 10 shows that the when the flame is detected by the IR flame sensors the pump Nossal is aimed at that angle

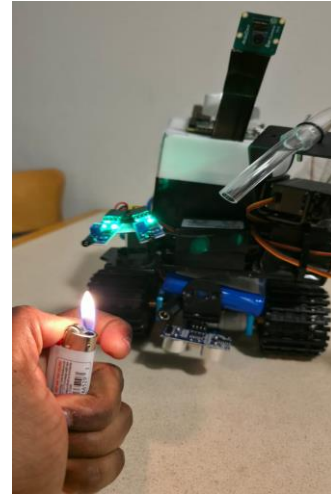


Figure. 10. Flame sensor and pump Nossal Aiming

E. Ultrasonic Staircase protection

The ultrasonic sensor reliably detected drop-offs and prevented falls in all the test trials. Figure 11 illustrates that the robot identifies the drop off and stops the motor immediately and protects the robot from drop off.



Figure. 11. Drop off protection using ultrasonic sensor

V. DISCUSSION

The system demonstrates that combining gesture-based control with sensor-based safety and vision-based monitoring yields a flexible and robust firefighting robot architecture. Separating monitoring (YOLO) from control (gestures + IR flame sensor + ultrasonic) simplifies system design and reduces unintended autonomous behavior. Limitations include the sensitivity of gesture recognition to lighting and the limited range of the IR flame sensor.

VI. CONCLUSION

This paper presented a gesture-controlled firefighting robot integrating TensorFlow Lite gesture recognition,

MQTT communication, YOLO-based monitoring, an IR flame sensor with servo-based pump aiming, and ultrasonic staircase protection. The system achieves reliable gesture recognition, low-latency control, effective fire targeting, and safe operation in indoor environments. The modular architecture makes it suitable for further research in human–robot interaction and safety-critical robotics.

REFERENCES

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018
- [2] S. Mitra and T. Acharya, “Gesture recognition: A survey,” *IEEE Trans. Syst., Man, Cybern.*, vol. 37, no. 3, pp. 311–324, 2007.
- [3] Google MediaPipe Documentation, “Hands Solution,” 2024.
- [4] MQTT.org, “MQTT Version 3.1.1 Specification,” 2023
- [5] TensorFlow Lite, “Model Optimization and Deployment Guide,” 2024.
- [6] A. M. Erkmen, “Embedded control systems in mobile robotics,” *IEEE Robot. Autom. Mag.*, vol. 15, no. 1, pp. 48–56, 2008.
- [7] Raspberry Pi Foundation, “Raspberry Pi Camera Module Documentation,” 2024.
- [8] HC-SR04 Ultrasonic Sensor Datasheet, 2023.
- [9] J. Borenstein and Y. Koren, “Real-time obstacle avoidance for mobile robots,” *IEEE Trans. Robot. Autom.*, 1991.
- [10] YOLOv8 Documentation, Ultralytics, 2024.
- [11] Z. Cao et al., “Real-time human detection for safety-critical robotics,” *IEEE Access*, 2021.
- [12] Flame Sensor Datasheet, Keyes, 2023